AI ASSISTED CODING
Assignment-22.5

Name: Mohammed Rahil Raza

Ht.no: 2303A51433

Bt.no:21

Task 1 – Fairness in AI Code Generation

• Provide AI with tasks involving different user groups (e.g., age, gender).

• Compare outputs for fairness.

• Discuss potential bias and its impact.

Code screenshot:

```python
user_groups = {
    "age": ["child", "adult", "senior"],
    "gender": ["male", "female", "non-binary"]
}
# Function to generate AI output based on user group
def generate_output(user_group):
    # Simulate AI output based on user group
    if user_group == "child":
        return "This is a fun game for kids!"
    elif user_group == "adult":
        return "This is a useful tool for adults."
    elif user_group == "senior":
        return "This is a helpful resource for seniors."
    elif user_group == "male":
        return "This product is designed for men."
    elif user_group == "female":
        return "This product is designed for women."
    elif user_group == "non-binary":
        return "This product is designed for everyone."
    else:
        return "User group not recognized."
# Compare outputs for fairness
for group in user_groups["age"]:
    output = generate_output(group)
    print(f"Output for {group}: {output}")
for group in user_groups["gender"]:
    output = generate_output(group)
    print(f"Output for {group}: {output}")
# Discuss potential bias and its impact
# The outputs generated for different user groups show a clear bias in the language used. For example
# the outputs for "child", "adult", and "senior" use different adjectives that may reinforce stereotypes about these age groups. Similarly, the outputs
#for "male" and "female" use gender-specific language that may perpetuate gender stereotypes. The output for "non-binary" is more inclusive, but it may still be perceived as less specif
```

Output:

```
PS C:\Users\paruc\AppData\Local\Programs\Microsoft VS Code> & C:\User
Output for child: This is a fun game for kids!
Output for adult: This is a useful tool for adults.
Output for senior: This is a helpful resource for seniors.
Output for male: This product is designed for men.
Output for female: This product is designed for women.
Output for non-binary: This product is designed for everyone.
```

Task 2 – Transparency of AI Decisions

- Use AI to generate a recommendation system.
- Analyze whether the logic is explainable.

• Discuss the need for transparency in AI-generated code.

Aiac_task2.py > ...

```python
# Import necessary libraries
import numpy as np
import pandas as pd
import os
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
# File path
file_path = r'C:\Users\saisharan\Downloads\College files\ALL SEMESTERS\6TH SEM\AI assistant coding\22.5\task2.csv'
try:
    data = pd.read_csv(file_path)

    if data.empty or len(data.columns) == 0:
        raise ValueError("Empty file")
except:
    print("⚠️ CSV file is empty or corrupted. Creating new dataset...")

    # Create sample dataset
    data = pd.DataFrame({
        'age': [25, 30, 22, 35, 28, 40, 33, 27],
        'income': [50000, 60000, 45000, 80000, 52000, 90000, 70000, 48000],
        'location': [1, 2, 1, 3, 2, 3, 2, 1],
        'preference': [0, 1, 0, 1, 1, 1, 1, 0]
    })
    # Save new CSV
    data.to_csv(file_path, index=False)
    print("✅ New dataset created successfully!")

# ------------------------------
# Step 2: Preprocess data
# ------------------------------
X = data.drop('preference', axis=1)
y = data['preference']

# ------------------------------
# Step 3: Train-test split
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# Step 4: Train model
# ------------------------------
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# ------------------------------
# Step 5: Predictions
```

```python
# ------------------------------
X = data.drop('preference', axis=1)
y = data['preference']

# ------------------------------
# Step 3: Train-test split
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# Step 4: Train model
# ------------------------------
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# ------------------------------
# Step 5: Predictions
# ------------------------------
predictions = model.predict(X_test)

# ------------------------------
# Step 6: Explainability
# ------------------------------
feature_importances = model.feature_importances_

print("\n🔍 Feature Importance:")
for feature, importance in zip(X.columns, feature_importances):
    print(f'Feature: {feature}, Importance: {round(importance, 3)}')

# ------------------------------
# Step 7: Transparency Discussion
# ------------------------------
print("\n📌 Transparency in AI-generated code is crucial because:")
print("1. Trust: Users trust systems they understand.")
print("2. Accountability: Decisions can be traced.")
print("3. Bias Detection: Helps identify unfair patterns.")
print("4. Debugging: Easier to fix errors.")
print("5. Compliance: Required in many industries."
```

Output:

```
✅ New dataset created successfully!

🔍 Feature Importance:
Feature: age, Importance: 0.367
Feature: income, Importance: 0.357
Feature: location, Importance: 0.276

📌 Transparency in AI-generated code is crucial because:
1. Trust: Users trust systems they understand.
2. Accountability: Decisions can be traced.
3. Bias Detection: Helps identify unfair patterns.
4. Debugging: Easier to fix errors.
5. Compliance: Required in many industries.
```

Task 3 – Data Misuse and Ethical Risks

• Ask AI to process datasets containing user information.

• Identify risks of misuse or leakage.

• Propose safeguards.

Code screenshot:

```
# Import necessary libraries
import pandas as pd
# Load the dataset containing user information
# For demonstration purposes, we will create a sample dataset
data = {
    'UserID': [1, 2, 3],
    'Name': ['Sharan', 'Lokesh', 'Rahul'],
    'Email': ['sharan@example.com', 'lokesh@example.com', 'rahul@example.com']

}
df = pd.DataFrame(data)
# Display the dataset
print("Dataset containing user information:")
print(df)
# Identify risks of misuse or leakage
print("\nRisks of misuse or leakage:")
print("1. Unauthorized access to user information.")
print("2. Data breaches leading to exposure of sensitive information.")

# Propose safeguards
print("\nProposed safeguards:")
print("1. Implement strong access controls to restrict who can view or modify the data.")
print("2. Use encryption to protect sensitive information both in transit and at rest.")
print("3. Regularly audit and monitor access to the dataset to detect any unauthorized activities.")
print("4. Educate employees and users about data privacy and security best practices.")
```

Output:

```
Dataset containing user information:
   UserID    Name               Email
0       1  Sharan  sharan@example.com
1       2  Lokesh  lokesh@example.com
2       3   Rahul   rahul@example.com

Risks of misuse or leakage:
1. Unauthorized access to user information.
2. Data breaches leading to exposure of sensitive information.

Proposed safeguards:
1. Implement strong access controls to restrict who can view or modify the data.
2. Use encryption to protect sensitive information both in transit and at rest.
3. Regularly audit and monitor access to the dataset to detect any unauthorized activities.
4. Educate employees and users about data privacy and security best practices.
```

Task 4 – AI Hallucination in Coding

• Ask AI to generate complex algorithms.

• Verify correctness of output.

• Identify incorrect or misleading code.

Code screenshot:

```python
# Example of AI-generated code for a complex algorithm (e.g., Dijkstra's algorithm)
def dijkstra(graph, start):
    import heapq
    queue = []
    heapq.heappush(queue, (0, start))
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0
    while queue:
        current_distance, current_vertex = heapq.heappop(queue)
        if current_distance > distances[current_vertex]:
            continue
        for neighbor, weight in graph[current_vertex].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(queue, (distance, neighbor))
    return distances
# Example of verifying correctness of the output
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'A': 1, 'C': 2, 'D': 5},
    'C': {'A': 4, 'B': 2, 'D': 1},
    'D': {'B': 5, 'C': 1}
}
start_vertex = 'A'
distances = dijkstra(graph, start_vertex)
print(distances)  # Expected output: {'A': 0, 'B': 1, 'C': 3, 'D': 4}
# Example of identifying incorrect or misleading code
def incorrect_dijkstra(graph, start):
    # This code has a logical error in the distance calculation
    import heapq
    queue = []
    heapq.heappush(queue, (0, start))
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0
    while queue:
        current_distance, current_vertex = heapq.heappop(queue)
        if current_distance > distances[current_vertex]:
            continue
        for neighbor, weight in graph[current_vertex].items():
            # Incorrect distance calculation (should be current_distance + weight)
            distance = weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(queue, (distance, neighbor))
    return distances
```

```python
# Testing the incorrect code
incorrect_distances = incorrect_dijkstra(graph, start_vertex)
print(incorrect_distances)  # This will likely produce incorrect
```

Output:

```
LOKESH/Downloads/College files/ALL SEMESTERS/6TH SEM/AI
Correct distances  : {'A': 0, 'B': 1, 'C': 3, 'D': 4}
Incorrect distances: {'A': 0, 'B': 1, 'C': 1, 'D': 1}
PS C:\Users\LOKESH\Downloads\College files\ALL SEMESTERS
```

Task 5– Responsibility in AI Deployment

• Scenario: AI code is deployed in a banking system.

• Identify risks and accountability.

• Propose ethical guidelines.

Output:

In the deployment of AI code in a banking system, several risks and accountability issues arise.

1. Risks:

a. Data Privacy: AI systems may access and process sensitive customer data, leading to potential breaches of privacy if not handled properly.

b. Bias and Discrimination: AI algorithms may inadvertently perpetuate biases present in the training data, resulting in unfair treatment of certain customer groups.

c. Security Vulnerabilities: AI systems can be targeted by cyberattacks, which may compromise the integrity and security of the banking system.

d. System Failures: AI systems may malfunction or produce incorrect outputs, leading to financial losses or damage to the bank's reputation.

2. Accountability:

a. Developers: AI developers are responsible for creating and testing the AI code to ensure it functions correctly and ethically.

b. Bank Management: The bank's management is accountable for overseeing the deployment of AI systems and ensuring they align with ethical standards and regulatory requirements.

c. Regulatory Bodies: Regulatory bodies are responsible for setting guidelines and enforcing compliance to ensure that AI systems in banking operate within legal and ethical boundaries.

3. Ethical Guidelines:

a. Transparency: AI systems should be transparent in their decision-making processes, allowing customers and regulators to understand how decisions are made.

b. Fairness: AI algorithms should be designed to minimize bias and ensure fair treatment of all customers, regardless of their background or characteristics.

c. Data Protection: Banks should implement robust data protection measures to safeguard customer information and comply with privacy regulations.

d. Accountability: Clear lines of accountability should be established, with defined roles and responsibilities for developers, management, and regulatory bodies to ensure ethical deployment of AI systems in banking.

e. Continuous Monitoring: AI systems should be continuously monitored for performance and ethical compliance, with mechanisms in place to address any issues that arise promptly.

f. Customer Consent: Banks should obtain explicit consent from customers before using their data for AI processing and provide options for customers to opt-out if they choose.

g. Regular Audits: Regular audits should be conducted to assess the ethical implications and performance of AI systems, ensuring they adhere to established guidelines and standards.

By addressing these risks and implementing ethical guidelines, banks can ensure responsible deployment of AI systems, fostering trust and safeguarding the interests of their customers.

Task 6 – Ethical Issues in Automation

• Use AI to automate a decision-making system (e.g., hiring filter).

• Analyze fairness and ethical concerns.

• Suggest improvements.

Code screenshot:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
# Sample dataset (you can replace this with your actual CSV data)
data = pd.DataFrame({
    'experience': [5, 3, 10, 2, 7],
    'education': ['Bachelors', 'Masters', 'PhD', 'Bachelors', 'Masters'],
    'skills': ['Python, SQL', 'Java, C++', 'Python, R', 'JavaScript, HTML', 'Python, Java'],
    'hired': [1, 0, 1, 0, 1]  # Target variable (1 for hired, 0 for not hired)
})
# Preprocess the data (handle missing values, encode categorical variables, etc.)
# For simplicity, let's assume the dataset is already clean and ready for modeling
# Encode categorical variables
data['education'] = data['education'].map({'Bachelors': 0, 'Masters': 1, 'PhD': 2})
data['skills'] = data['skills'].apply(lambda x: 1 if 'Python' in x else 0)  # Example encoding for skills
# Split the data into features and target variable
X = data.drop('hired', axis=1)  # Features

y = data['hired']  # Target variable
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train a Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred = model.predict(X_test)
# Evaluate the model
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
# Analyze fairness and ethical concerns
# Check for bias in the model (e.g., Analyze feature importance to see if certain features are disproportionately influencing the model)
feature_importances = model.feature_importances_
feature_names = X.columns
for name, importance in zip(feature_names, feature_importances):
    print(f'Feature: {name}, Importance: {importance}')
# Suggest improvements
# 1. Collect more diverse and representative data to reduce bias.
# 2. Implement fairness-aware algorithms that adjust for bias in the training data.
# 3. Regularly audit the model's predictions for fairness and adjust as necessary.
# 4. Provide transparency in the decision-making process to allow for accountability.
# 5. Involve stakeholders in the development and deployment of the model to ensure ethical considerations are addressed.
```

Output:

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1

C:\Users\LOKESH\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\metrics\_classification.py:534: UserWarning: A sin
gle label was found in 'y_true' and 'y_pred'. For the confusion matrix to have the correct shape, use the 'labels' parameter to pass
 all known labels.
  warnings.warn(
[[1]]
Feature: experience, Importance: 0.44920634920634916
Feature: education, Importance: 0.2
Feature: skills, Importance: 0.3507936507936509
```

Task 7 – AI and Intellectual Property

• Generate code using AI for a known problem.

• Check similarity with existing solutions.

• Discuss plagiarism and copyright concerns.

Code screenshot:

```python
# Task 7 - AI and Intellectual Property
import difflib
# Function to generate code using AI (simulated here)
def generate_code(problem):
    # Simulating AI-generated code for a known problem (e.g., Fibonacci sequence)
    if problem == "Fibonacci":
        return """
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        seq = [0, 1]
        for i in range(2, n):
            seq.append(seq[i-1] + seq[i-2])
        return seq
"""

# Function to check similarity with existing solutions
def check_similarity(generated_code, existing_code):
    similarity = difflib.SequenceMatcher(None, generated_code, existing_code).ratio()
    return similarity
# Simulating existing solution for Fibonacci sequence
existing_solution = """
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        seq = [0, 1]
        for i in range(2, n):
            seq.append(seq[i-1] + seq[i-2])
        return seq
"""
# Generate code using AI
problem = "Fibonacci"
generated_code = generate_code(problem)
# Check similarity with existing solution
similarity = check_similarity(generated_code, existing_solution)
print(f"Generated Code:\n{generated_code}")
print(f"Similarity with existing solution: {similarity:.2f}")
# Discuss plagiarism and copyright concerns
if similarity > 0.8:
    print("The generated code is very similar to the existing solution. This raises concerns about plagiarism and copyright infringement.")
else:
    print("The generated code is sufficiently different from the existing solution, but it's important to always give credit to original sources and avoid copying code without permission.")
```

Output:

```
Generated Code:

def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        seq = [0, 1]
        for i in range(2, n):
            seq.append(seq[i-1] + seq[i-2])
        return seq

Similarity with existing solution: 1.00
The generated code is very similar to the existing solution. This raises concerns about plagiarism and copyright infringement.
```